

Swagger / OpenAPI in a .NET API

Documenting, testing and consuming SimpleStudentApi

Group PSe-5

Autumn 2026

Lecture + practice

Lecturer: Tadas Bulis

What we cover today

Five blocks — the last one is your practice

1

What Swagger and OpenAPI are

Terms, and why an API needs a contract

2

Adding Swagger to the API

One package, four lines of setup, then describing endpoints

3

Working in Swagger UI

Read, try out, see errors, copy curl

4

Using the contract in the frontend

A generated typed client instead of hand-written HTTP calls

5

Practice and your documentation

Lab tasks, and where OpenAPI fits in the project documentation

OpenAPI is the contract between the backend and everyone who calls it: one machine-readable file that people read, tools test, and generators turn into client code.

LT: OpenAPI – API kontraktas · Swagger UI – interaktyvi jo peržiūra

Four names you will hear

They are often mixed up — here is who is who

1 OpenAPI Specification

The standard (OAS 3.x) for describing HTTP APIs in JSON or YAML: paths, parameters, request bodies, responses, schemas. Vendor-neutral, maintained under the Linux Foundation.

2 Swagger

The original name of the specification (until 2015) and today SmartBear's family of tools around it. Swagger UI is the web page that renders an OpenAPI file.

3 Swashbuckle.AspNetCore

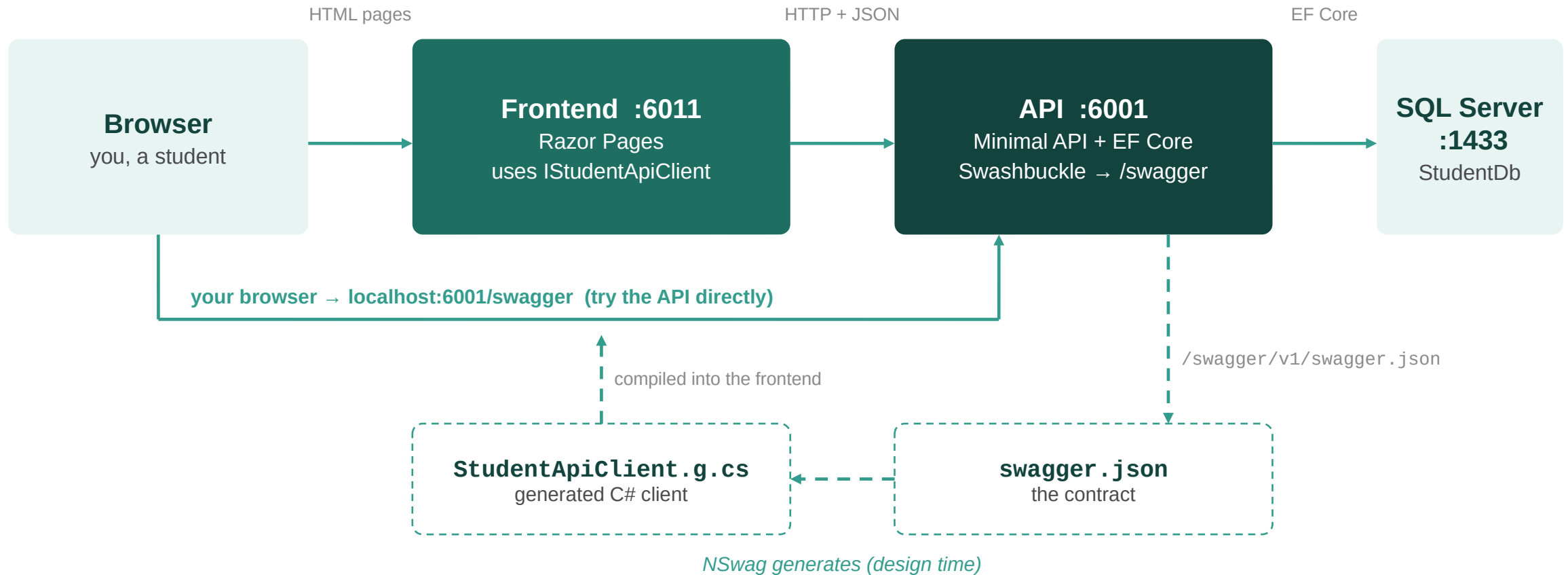
The NuGet package we use in the API: it reads our .NET endpoints, generates swagger.json and hosts Swagger UI at /swagger.

4 NSwag

A generator that reads swagger.json and writes a typed C# (or TypeScript) client. We use it in the frontend.

Swagger in our Docker lab

Same containers as before — two new paths



Adding Swagger: 1 package, 4 lines

SimpleStudentApi/Program.cs

1

Install the package

Swashbuckle.AspNetCore from NuGet

2

Register the generator

AddEndpointsApiExplorer + AddSwaggerGen

3

Serve the JSON

app.UseSwagger() → /swagger/v1/swagger.json

4

Serve the UI

app.UseSwaggerUI() → /swagger

```
dotnet add package Swashbuckle.AspNetCore
```

```
// SimpleStudentApi.csproj
<PackageReference Include="Swashbuckle.AspNetCore"
                  Version="10.2.3" />

// Program.cs
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(o =>
{
    o.SwaggerDoc("v1", new OpenApiInfo
        { Title = "SimpleStudentApi", Version = "v1" });
});

var app = builder.Build();

app.UseSwagger();           // /swagger/v1/swagger.json
app.UseSwaggerUI();        // /swagger
```

Metadata turns code into documentation

Each call only adds information to swagger.json — runtime behaviour does not change

```
// handler: validate → save → 201 Created
students.MapPost("/", CreateStudent)
    .WithName("CreateStudent")
    .WithTags("Students")
    .WithSummary("Create a new student")
    .WithDescription("The id is ignored ...")
    .Produces<Student>(201)
    .ProducesValidationProblem();    // 400

students.MapGet("/{id:int}", GetStudent)
    .WithName("GetStudent")
    .Produces<Student>(200)
    .Produces(404);
```

Code	Appears in Swagger as
.WithName("CreateStudent")	operationId → method CreateStudentAsync() in the generated client
.WithTags("Students")	group heading "Students"
.WithSummary / .WithDescription	text next to and under the endpoint
.Produces<Student>(201)	response code + JSON schema
.ProducesValidationProblem()	400 response with field errors
.ExcludeFromDescription()	hides an endpoint (our "/" redirect)

Comments become examples

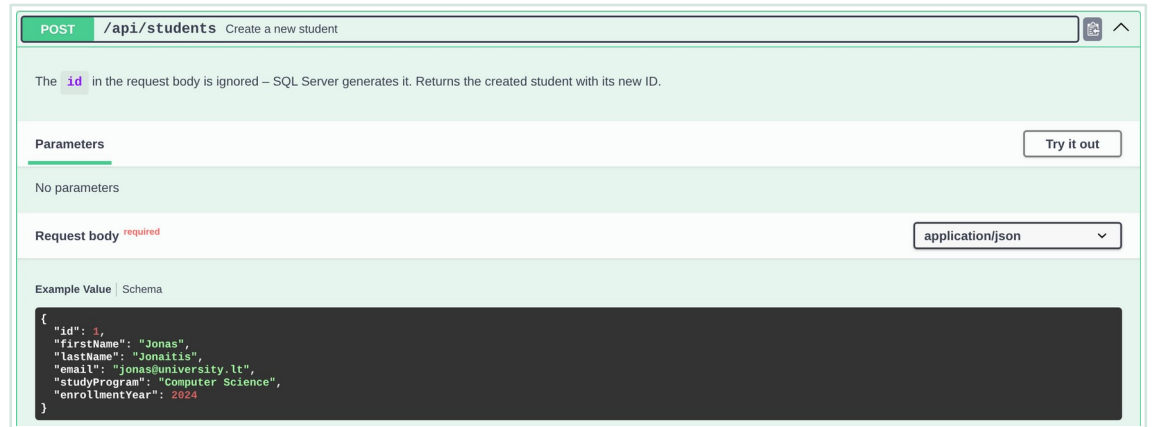
/// <summary> and <example> on the model → schema and example value in Swagger UI

```
// SimpleStudentApi.csproj
<GenerateDocumentationFile>true</GenerateDocumentationFile>

// Program.cs → inside AddSwaggerGen(o => ...)
o.IncludeXmlComments(xmlFile);

// Models/Student.cs
/// <summary>A student (table Students)</summary>
public class Student
{
    /// <example>Jonas</example>
    public string FirstName { get; set; } = "";

    /// <example>2024</example>
    public int EnrollmentYear { get; set; }
}
```



Result: description and an example request body in Swagger UI



Common mistake: IncludeXmlComments without GenerateDocumentationFile — the XML file does not exist and the API fails at startup.

http://localhost:6001/swagger

Every endpoint, grouped by tag, colour-coded by HTTP method

 **Swagger**
supported by SMARTBEAR

Select a definition

SimpleStudentApi v1



SimpleStudentApi v1 OAS 3.0

</swagger/v1/swagger.json>

Student CRUD API (.NET 8 Minimal API + EF Core + SQL Server) used in the Heterogeneous Systems Docker lab.

Faculty

GET

/api/faculty

Faculty → groups → students

Info

GET

/api/info

Service information

Students

GET

/api/students

List all students

POST

/api/students

Create a new student

GET

/api/students/{id}

Get one student by ID

PUT

/api/students/{id}

Update an existing student

DELETE

/api/students/{id}

Delete a student

Tags

group endpoints (WithTags)

Colours

GET blue · POST green · PUT orange · DELETE red

Contract link

/swagger/v1/swagger.json — the raw OpenAPI file

Schemas

every request and response model

Test the API without writing a client

Expand → Try it out → edit the body → Execute

1

Expand POST /api/students

description, body schema and responses appear

2

Click "Try it out"

the example body becomes editable

3

Send invalid data

empty first name, year 1990 → Execute

4

Read the result

400, field errors, headers and a ready curl command

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:6002/api/students' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "firstName": "",
    "lastName": "Jonaitis",
    "email": "not-an-email",
    "studyProgram": "Computer Science",
    "enrollmentYear": 1990
  }'
```

Request URL

http://localhost:6002/api/students

Server response

Code	Details
------	---------

400	Error: Bad Request
-----	--------------------

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "firstName": [
      "First name is required."
    ],
    "email": [
      "A valid email is required."
    ],
    "enrollmentYear": [
      "Enrollment year must be between 2000 and 2100."
    ]
  }
}
```

Download

Swagger UI sends real requests — what you POST is really saved in SQL Server.

One document, six uses

Why the ten minutes of setup pay off

1 Living documentation

Generated from code, so it is never out of date

2 Manual testing

Try every endpoint in the browser — no Postman needed for the basics

3 Error contract

201 / 400 / 404 responses are documented and visible

4 curl and request URL

Copy the exact request into a terminal, a script or a bug report

5 Client generation

NSwag or OpenAPI Generator create C#, TypeScript, Java... clients

6 Interface specification

Attach swagger.json to your project documentation

Two URLs, two audiences

Containers talk by service name; your browser talks through the host port

Inside the Docker network

- **Frontend → API: `http://simplestudentapi:8080`**
- Set by `ApiSettings__BaseUrl`
- Used by the generated client (server side)

From your computer

- **Browser → API: `http://localhost:6001`**
- Set by `ApiSettings__PublicUrl`
- Used for the Swagger link and Swagger UI

Swagger is enabled in every environment here because the containers run as Production. In a real product: Development only, or behind authentication.

Three ways to call the API

The two labs show the first two side by side

A Hand-written HttpClient

```
GetFromJsonAsync<ApiStudentResponse>("/api/getStudent")
```

- + simple, no extra tools
- URLs and DTOs typed by hand; typos found at runtime

Lab: ...with-additional-endpoint-and-page

B Generated typed client (NSwag)

```
await _api.GetStudentsAsync()
```

- + methods and models come from swagger.json; the compiler checks the contract
- regenerate after API changes

Lab: ...plus-mssql

C JavaScript in the browser

```
fetch("http://localhost:6001/api/students")
```

- + SPA frameworks (React, Angular) — generate a TypeScript client the same way
- the API must allow CORS

Not in our labs

Generate the client from swagger.json

SimpleStudentFrontend/ApiClient/generate-client.sh (Windows: .ps1)

1

Start the API

docker compose up (or dotnet run)

2

Run generate-client

downloads swagger.json and runs NSwag via npx
(needs Node.js)

3

Rebuild the frontend

docker compose up --build

```
npx nswag openapi2csclient /runtime:Net80 \  
  /input:swagger.json \  
  /output:StudentApiClient.g.cs \  
  /namespace:SimpleStudentFrontend.ApiClient \  
  /className:StudentApiClient \  
  /jsonLibrary:SystemTextJson \  
  /generateClientInterfaces:true \  
  /useBaseUrl:false
```

Result: IStudentApiClient, StudentApiClient and the models Student, InfoResponse, FacultyResponse, ApiException — about 1 000 lines you never write by hand.

Register once, inject everywhere

Program.cs, then before vs after in Students.cshtml.cs

```
// Program.cs
builder.Services.AddHttpClient<IStudentApiClient, StudentApiClient>(c =>
    c.BaseAddress = new Uri(apiBaseUrl));
```

Before (hand-written)

```
var client = _httpClientFactory
    .CreateClient("StudentApi");

Students = await client
    .GetFromJsonAsync<List<StudentDto>>(
        "/api/students");

await client.DeleteAsync(
    $"/api/students/{id}");
```

After (generated client)

```
// IStudentApiClient _api is injected

Students = (await _api.GetStudentsAsync())
    .ToList();

await _api.DeleteStudentAsync(id);
```

Wrong URL or field name → compile error instead of a 404 at runtime

API errors become typed exceptions

The 400 documented in Swagger arrives in C# as `ApiException<HttpValidationProblemDetails>`

```
try
{
    await _api.CreateStudentAsync(Input.ToApi());
    return RedirectToPage("/Students");
}
catch (ApiException<HttpValidationProblemDetails> ex)
{
    // 400 - field errors from the API
    ErrorMessage = string.Join("; ",
        ex.Result.Errors.SelectMany(e => e.Value));
}
catch (ApiException ex) when (ex.StatusCode == 404)
{
    ErrorMessage = "Student not found";
}
```

Simplified — see `ApiClient/ApiErrors.cs`

[Info](#) [Students](#) [API docs \(Swagger\)](#) ↗

Add New Student

Error: Validation failed: Enrollment year must be between 2000 and 2100.

First Name

Last Name

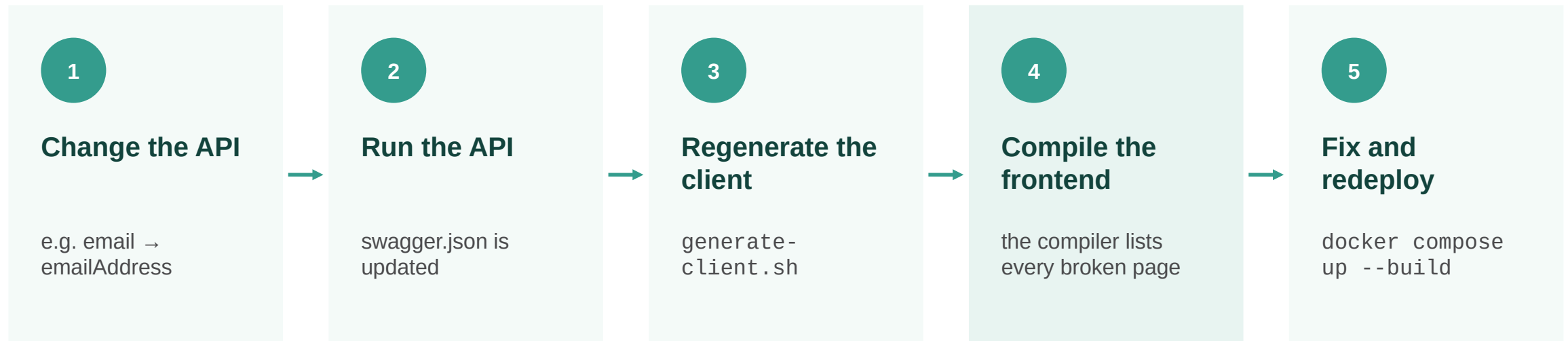
Email

Study Program

Enrollment Year

When the API changes

The contract makes breaking changes visible before users see them



Without the generated client, step 4 would be a runtime bug found by a user.

Lab tasks (60 min)

Work in student-docker-lab-plus-mssql

1

Run it and explore

docker compose up; open /swagger; create 3 students with Try it out

2

Break it on purpose

POST invalid data; compare the 400 in Swagger and in the frontend form

3

Add an endpoint

GET /api/students/search?program=... with WithName, WithSummary, Produces

4

Regenerate and use

run generate-client; add a search box to the Students page using SearchStudentsAsync

5

Document it

export swagger.json and add it to your project documentation as the interface specification

OpenAPI as the interface specification

How it maps to the standards from the previous lectures

1 ISO/IEC/IEEE 42010

Interface / information view:
swagger.json is the formal description of
the API connector between components.

2 ISO/IEC 25010

Compatibility → interoperability: a
published contract is evidence that the
systems can exchange data.

3 European Interoperability Framework

Technical and semantic interoperability:
an open standard (OAS) and a shared
data model (schemas).

In your documentation: the OpenAPI file (appendix), a Swagger UI screenshot, and a table of endpoints with status codes.

Swagger turns your API into a contract you can read, test and compile against.

- Swashbuckle → swagger.json + /swagger
- Metadata and XML comments → useful documentation
- NSwag → typed client; the compiler checks the contract